Universidad Autónoma de Tamaulipas
Facultad de Ingeniería Tampico

VERDAD, BELLEZA, PROBIDAD

INGENIERÍA TAMPICO

ASIGNATURA
Programación de interfaces y puertos
6to. Semestre – Grupo “I”
2025 -1

TRABAJO
Desarrollo de Prácticas y Proyectos

UNIDAD
1 - Modelos de interacción computacional

Docente: Dr. García Ruiz Alejandro H.

Integrante del Equipo	Nivel de Participación
Rosales Pereles Denisse Ariadna	33.33%
Morales Urrutia Javier Antonio	33.33%
Vicente Bautista Roberto Nicolas	33.33%
Total:	100%

Aniversario 75 UAT 1950-2025

Índice

Repositorio(s) de Proyectos 2

P1. Recopilación y almacenamiento de datos 2

Descripción del proyecto 2

Introducción 2

Componentes para el desarrollo del proyecto 2

Desarrollo 3

Conclusiones 10

Repositorio(s) de Proyectos

Proyecto	Repositorio
Proyecto U1	https://github.com/JavierMoralesU/PIP/blob/master/Unidad%201/Ejercicios/Proyecto_Unidad1.py

P1. Recopilación y almacenamiento de datos

Descripción del proyecto

En este proyecto se desea realizar un sistema con Python GUI que permitirá realizar la recopilación y almacenamiento de datos, así como también el cálculo de la "Tendencia central" (Media, mediana, moda), y las medidas de "Dispersión" (Valor menor, valor mayor, desviación estándar, varianza). En este sistema, el usuario podrá ingresar valores numéricos para su después calculo, por ello, el usuario podrá guardar la lista de números ingresada, así como también los cálculos, estos resultados del análisis serán almacenados en un archivo y por ello poder ser cargados y visualizados desde el sistema.

Introducción

En la actualidad, la cantidad de datos que se generan diariamente es abrumadora, por lo que es necesaria la recopilación y análisis de datos. Tanto en el ámbito empresarial como en el personal, la recopilación y análisis de datos se ha vuelto fundamental para la toma de decisiones efectivas y precisas.

Para facilitar este proceso, se desarrollará un Sistema con interfaz gráfica en Python GUI, el cual permitirá a los usuarios ingresar, almacenar y analizar datos numéricos de manera eficiente. Este sistema no solo brindara la funcionalidad de ingresar, guardar y cargar datos, sino que también calculara medidas estadísticas clave.

Componentes para el desarrollo del proyecto

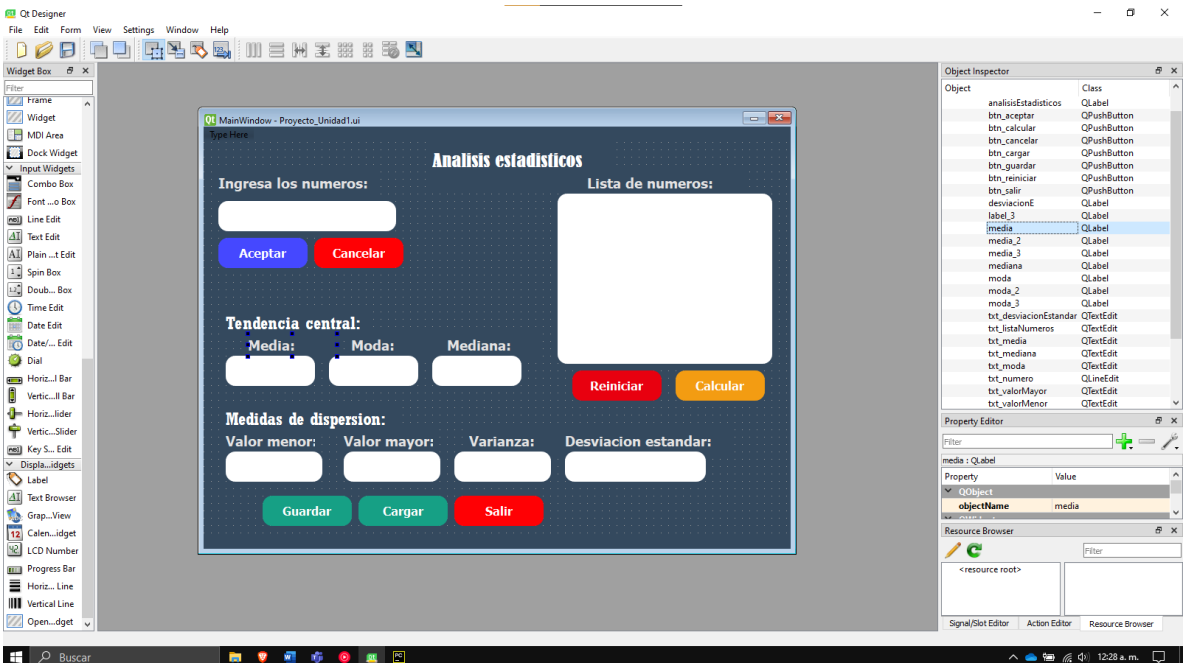
1. **QLabel:** Muestra textos descriptivos en la interfaz, como títulos, etiquetas de campos y resultados
2. **QTextEdit:** Permite al usuario ingresar una lista de números para su posterior análisis, también se usará para mostrar los resultados calculados.

3. **QPushButton:** Se empleará para realizar diversas acciones dentro del sistema, por ejemplo:

- Aceptar: Capturar la lista de números ingresados
- Cancelar: Borra el último número ingresado por el usuario
- Calcular: Obtener los resultados de la tendencia central y dispersión
- Reiniciar: Borrar los resultados y la lista de números a un nuevo archivo
- Guardar: Guardar los números ingresados y los cálculos en un archivo
- Cargar: La carga de los datos almacenados en el archivo guardado previamente
- Salir: Salida del sistema

Desarrollo

Creación de la interfaz en Qt Designer:



Se agrega un conjunto de componentes para un sistema eficiente, fácil de usar y entendible, asegurando que el usuario pueda interactuar intuitivamente con la interfaz para el análisis de datos.

Incluye componentes como: QLabel, QPushButton, QTextEdit.

Object Inspector

Object	Class
centralwidget	QWidget
analisisEstadisticos	QLabel
btn_aceptar	QPushButton
btn_calcular	QPushButton
btn_cancelar	QPushButton
btn_cargar	QPushButton
btn_guardar	QPushButton
btn_reiniciar	QPushButton
btn_salir	QPushButton
desviacionE	QLabel
label_3	QLabel
media	QLabel
media_2	QLabel
media_3	QLabel
mediana	QLabel
moda	QLabel
moda_2	QLabel
moda_3	QLabel
txt_desviacionEstandar	QTextEdit
txt_listaNumeros	QTextEdit
txt_media	QTextEdit
txt_mediana	QTextEdit
txt_moda	QTextEdit

Una vez diseñado la interfaz en Qt Designer, se guarda el archivo ".ui", para su posterior funcionalidad en código.

```python
from statistics import variance, stdev
#import mean
from PyQt5 import uic, QtWidgets

qtCreatorFile = "Proyecto_Unidad1.ui"  # Nombre del archivo aquí.
Ui_MainWindow, QtBaseClass = uic.loadUiType(qtCreatorFile)

class MyApp(QtWidgets.QMainWindow, Ui_MainWindow):
    def __init__(self):
        super().__init__()
        self.setupUi(self)

        #ingresa los numeros
        self.btn_aceptar.clicked.connect(self.aceptar)
        self.btn_cancelar.clicked.connect(self.cancelar)

        #Lista de numeros
        self.btn_reiniciar.clicked.connect(self.reiniciar)
        self.btn_calcular.clicked.connect(self.calcular)

        #Tendencia central y medidas de dispersion
        self.btn_guardar.clicked.connect(self.guardar)
        self.btn_cargar.clicked.connect(self.cargar)
        self.btn_salir.clicked.connect(self.salir)
        self.numeros = []

        self.ruta_archivo_Numeros= "../../Archivos/numerosAnalisis.csv"
        self.ruta_archivo_Resultados= "../../Archivos/resultadosAnalisis.txt"
        self.guardado = True
    def aceptar(self):  1 usage
```

Implementación del código en Python

En la clase principal, le damos funcionalidad a los botones de la interfaz hecha previamente (del archivo .ui). Por ello, se crea una ruta de archivo para almacenar la lista de números que puede ser ingresada, y una ruta donde se almacenara un .txt con los datos estadísticos.

Además, se utiliza la biblioteca llamada: statistics, para el cálculo rápido de las tendencias centrales y de dispersión.

```python
    def aceptar(self):  1 usage
        num_texto=self.txt_numero.text().strip()

        try:
            num=float(num_texto)
            self.numeros.append(num)
            self.txt_numero.clear()
            self.mostrar_numeros()
            self.guardado = False
            self.cambiar_boton_guardar()

        except ValueError:
            QtWidgets.QMessageBox.warning(self, title "ERROR", text "Ingresa un numero valido")

    #ListaNumeros (CUADRO)
    def mostrar_numeros(self):  2 usages
            self.txt_listaNumeros.setText(str(self.numeros))
```

Le damos funcionalidad a cada método, por ejemplo, cuando el usuario ingresa un numero y le da clic en Aceptar, este número se extrae en tipo cadena y se hace la conversión a tipo numérico, por lo que el numero se guarda en la lista de números, un atributo de la clase y se usa el método mostrar_numeros para mostrar los números actuales en un cuadro de texto.

```python
    def cancelar(self):  1 usage
        self.numeros.pop()
        self.txt_numero.clear()
        self.mostrar_numeros()
        self.guardado = False
        self.msj( title "correcto",  txt: f"Ultimo numero eliminado")


    def reiniciar(self):  1 usage
        self.numeros=[]
        self.txt_listaNumeros.clear()
        self.txt_media.clear(), self.txt_mediana.clear(), self.txt_moda.clear()
        self.txt_valorMenor.clear(), self.txt_valorMayor.clear(), self.txt_varianza.clear(), self.txt_desviacionEstandar.clear()
        self.guardado = False
        self.msj( title "Correcto",  txt: f"Lista reiniciada...")
```

Creamos los demás métodos para darle función a los botones del diseño, en este caso también se le da función al botón de: “Cancelar”, su función principal es borrar el ultimo numero de la lista de números ingresada. Hablando de código, utilizamos el .pop() para sacar el último número y se usa mostrar_numeros, por lo que el cuadro de texto donde se envía la información también se actualiza.

Para el método reiniciar solo basta de conocer los números ingresados por el usuario que se guardan en self.numeros, basta con = [] para igualarlo a una “nueva” lista vacía.

```python
def calcular(self):
    #Tendencia central
    media=statistics.mean(self.numeros) #CALCULA LA MEDIA ARITMETICA
    self.txt_media.setText(str(media))

    mediana=statistics.median(self.numeros) #cacula mediana
    self.txt_mediana.setText(str(mediana))

    moda=statistics.mode(self.numeros) #CALCULA LA MODA
    self.txt_moda.setText(str(moda))

    #Medidas de dispersion
    valorMenor1=min(self.numeros)
    self.txt_valorMenor.setText(str(valorMenor1))

    valorMayor1=max(self.numeros)
    self.txt_valorMayor.setText(str(valorMayor1))

    varianza=variance(self.numeros)
    self.txt_varianza.setText(str(varianza))

    desviacionE=stdev(self.numeros)
    self.txt_desviacionEstandar.setText(str(desviacionE))
    self.msj( title "Correcto",  txt: f"Valores calculados.")
```

Además, para el cálculo que nos pide el ejercicio, solo basto de agregar palabras clave, como por ejemplo:
mean, median, mode, min, max, variance y stev, esto se agrega después de haber instalado la biblioteca “statistics”.

```python
def guardar(self):  1 usage
    archivo = open(self.ruta_archivo_Numeros, "w")  # "w"=write (escritura / "a" = append
    for num in self.numeros:
        archivo.write(str(num) + "\n")
    archivo.flush()
    archivo.close()
    self.msj( title "Correcto",  txt: f"Archivo guardado correctamente.")
    self.guardado = True
    self.cambiar_boton_guardar()
```

Le damos función al método guardar, su principal objetivo es guardar la lista de números dentro de "Archivos/numerosAnalisis.csv"

Mientras que los resultados estadísticos obtenidos se escriben en un archivo: “Archivos/resultadosAnalisis.txt”

```python
# Guardado de los resultados
Narchivo = open(self.ruta_archivo_Resultados, "w")
Narchivo.write("Para la serie de numeros:\n")
Narchivo.write(f"{self.numeros}\n\n")
Narchivo.write("Los resultados son los siguientes:\n")
media   = self.txt_media.toPlainText().strip()
mediana = self.txt_mediana.toPlainText().strip()
moda    = self.txt_moda.toPlainText().strip()
Vmenor = self.txt_valorMenor.toPlainText().strip()
Vmayor = self.txt_valorMayor.toPlainText().strip()
varianza = self.txt_varianza.toPlainText().strip()
desviacion = self.txt_desviacionEstandar.toPlainText().strip()
Narchivo.write(f"Media = {media}\n")
Narchivo.write(f"Mediana = {mediana}\n")
Narchivo.write(f"Moda = {moda}\n")
Narchivo.write(f"Valor Menor = {Vmenor}\n")
Narchivo.write(f"Valor Mayor = {Vmayor}\n")
Narchivo.write(f"Varianza = {varianza}\n")
Narchivo.write(f"Desviacion Estandar = {desviacion}\n")
Narchivo.flush()
Narchivo.close()
```

Resultando en archivos como estos:

numerosAnalisis.csv

```
2.0
4.0
10.0
4.0
5.0
```

resultadosAnalisis.txt

```
Para la serie de numeros:
[2.0, 4.0, 10.0, 4.0, 5.0]

Los resultados son los siguientes:
Media = 5.0
Mediana = 4.0
Moda = 4.0
Valor Menor = 2.0
Valor Mayor = 10.0
Varianza = 9.0
Desviacion Estandar = 3.0
```

```python
def cargar(self):  1 usage
    archivo = open(self.ruta_archivo_Numeros)
    contenido = archivo.readlines()  # lee el archivo completo

    nums=[]
    for i in contenido:
        nums.append(float(i))

    print(nums)
    self.numeros=nums
    self.msj( title "Correcto",  txt: f"Archivo cargado.")
    self.calcular()
    self.txt_listaNumeros.setText(str(self.numeros))
    self.guardado = True
    self.cambiar_boton_guardar()
```

Tenemos también el método: Cargar, su objetivo principal es cargar los números previamente almacenado en el archivo, visualizando los datos desde el sistema, dentro de el recuadro para la lista de números.

```python
def salir(self):
    self.close()

def msj(self, title, txt):
    m = QtWidgets.QMessageBox()
    m.setIcon(QtWidgets.QMessageBox.Information)
    m.setWindowTitle(title)
    m.setText(txt)
    m.exec_()

def cambiar_boton_guardar(self):  3 usages
    if self.guardado:
        self.btn_guardar.setStyleSheet(""" background-color: #16A085;  border-radius: 13px;color: rgb(255, 255, 255); """)

    else:
        self.btn_guardar.setStyleSheet(""" background-color: #D10A5A; border-radius: 15px;color: rgb(255, 255, 255);""")
```

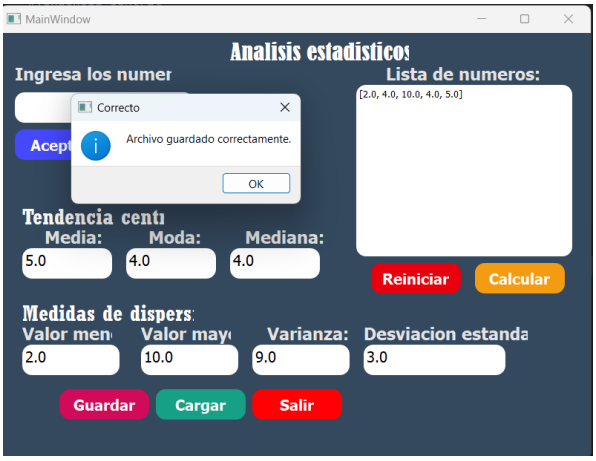
Al final nos encontramos con otros métodos de utilidad como es el msj para poder mostrar notificaciones al usuario a través de un QMessageBox, siendo usadas para mensajes de confirmación o para los try y Except usados con excepciones para casos de errores tales como el ingreso de letras y símbolos donde van números.

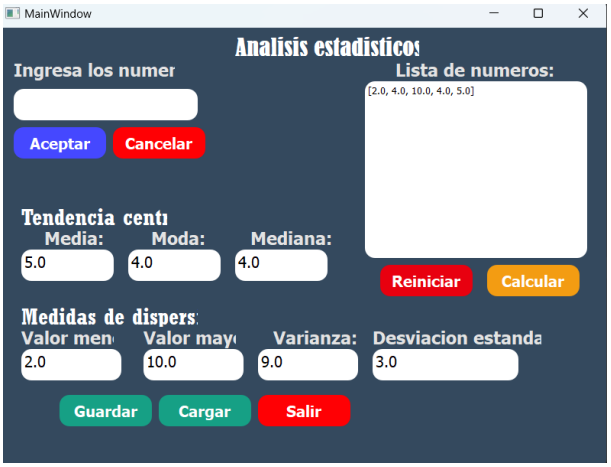
Y se usa el método cambiar_boton_guardar para tal como indica su nombre, cambiar el color del botón guardar y notificar que hay cambios por guardar, en un inicio el botón es verde pero al agregar o quitar números a la lista este pasara a rojo, una vez vuelto a guardar y el archivo haya sido actualizado vuelve al color verde.

Y finalmente, se agrega la funcionalidad al botón “Salir”, por lo que al darle clic, la aplicación se cierra.

Se ejecuta el programa correctamente, así como también el diseño visual hecho en Qt designer. Se hace la prueba con una lista de números ingresados por el usuario, lo cual al dar “Calcular” muestra los resultados en cada celda que corresponda.

Se le da en guardar y posteriormente los datos calculados, así como también la lista números queda almacenados en un archivo.





Se puede usar el botón de reiniciar para borrar la lista de números y resultados actuales, pero una vez guardado basta con darle a cargar para obtenerlos de nuevo, mostrarlos en la lista y llena los campos calculados.

Conclusiones

Este sistema representa una solución eficiente para la recopilación, almacenamiento y análisis de datos, proporcionando a los usuarios una herramienta confiable y accesible. Su flexibilidad y facilidad de uso lo convierten en una base sólida para futuras mejoras y aplicaciones más complejas.